



APPLICATION DEVELOPMENT PRACTICES

Dr.Lê Hùng Tiến
VLU



APPLICATION DEVELOPMENT PRACTICES

Closure and Reflection



Director's Overview

- Is Quality an Accident?
- Technical Debt



Is Quality an Accident? - 1

- If no, then what, specifically are you doing to ensure a quality product in a timely manner?



Is Quality an Accident? - 2

- Teamwork
 - Characteristics of high performance teams
 - Giving and receiving feedback
 - Personal responsibility
 - Values
 - What does your behavior say about you?



Is Quality an Accident? - 3

- Processes

- Why process and process measurement are important
- The processes we use to develop software
- How to document a process
- How much process is enough



Is Quality an Accident? - 4

- Inspection Process
 - Heavy versus lightweight processes to accomplish the same goals
 - Built an inspection process and inspected it
 - What does the data say?



Is Quality an Accident? - 5

- Configuration Management Processes and tools
 - Source control
 - Automated build
 - Automated unit test
 - Scheduled build and unit test
 - Worked with change management tools



Is Quality an Accident? - 6

- Fixing Defects
 - Defects as opportunities
 - Tips for finding defects
 - Tips for fixing defects
 - Finding defects versus preventing defects
 - Worked with change management tools



Is Quality an Accident? - 7

- Refactoring
 - Is your software's evolution:
 - A planned-for opportunity that is improving the internal quality of the software?
 - A haphazard activity that is continually degrading the product's quality?
 - Refactoring does NOT change the software's functionality
 - Complete one refactoring at a time
 - Worked with change management tools



Is Quality an Accident? - 8

- Analyzing and Estimating Requirements
 - Managing change to traditional requirements
 - Managing change to agile requirements
 - Worked with change management tools
 - Change estimation



Is Quality an Accident? - 9

- Testing and Quality
 - Is quality an accident?
 - Who owns quality?
 - Characteristics of software quality
 - Quality approaches
 - Find defects
 - Prevent defects
 - Do Both
 - Testing approaches
 - Unit
 - Integration
 - Acceptance
 - System



Is Quality an Accident? - 10

- Customer Satisfaction
 - What is Customer Satisfaction?
 - Kano Model
 - What if there are problems?



Technical Debt

- The obligation that a software organization incurs when it chooses a design or construction approach that's expedient in the short term but that increases complexity and is more costly in the long term
 - Coined by Ward Cunningham



Two Kinds of Technical Debt

- Unintentional
 - A design turns out to be error prone
 - A programmer makes a mistake
- Intentional
 - The organization makes a conscious decision to optimize for the present rather than for the future



Example

- “You are developing a large application that requires, among other things, 5 reports.
 - Option 1: Design a set of custom report writer classes
 - This will make generation of additional reports easier later on
 - Option 2: Write the reports using your database's built-in report generator instead
 - This makes writing the initial set of reports easier (mostly), but once that initial "starter kit" of reports are done, later reports will be more difficult”



Short and Long Term Debt

- “With real debt, a company will maintain both short-term and long-term debt
- Short term debt:
 - You use short-term debt to cover things like gaps between your receivables (payments from customers) and expenses (payroll)
 - You take on short term debt when you have the money, you just don't have it *now*
 - Short-term debt is expected to be paid off frequently
 - The technical equivalent:
 - Short-term debt is the debt that's taken on *tactically and reactively*, usually as a late-stage measure to get a specific release out the door”



Short and Long Term Debt

- “Long term debt:
 - Long term debt is the debt a company takes on *strategically and proactively*--investing in new capital equipment, like a new factory, or a new corporate campus
 - The technical equivalent:
 - "We don't think we're going to need to support a second platform for at least five years, so this release can be built on the assumption that we're supporting only one platform"
- The implication is that short-term debt should be paid off quickly, perhaps as the first part of the next release cycle, whereas long-term debt can be carried for a few years or longer”



Incurring Technical Debt

- “The cost of development work today is seen as more expensive than the cost will be in the future:
 - *Time to Market.* When time to market is critical, incurring an extra \$1 in development might equate to a loss of \$10 in revenue. Even if the development cost for the same work rises to \$5 later, incurring the \$1 debt now is a good business decision
 - *Preservation of Startup Capital.* In a startup environment you have a fixed amount of seed money, and every dollar counts. If you can delay an expense for a year or two you can pay for that expense out of a greater amount of money later rather than out of precious startup funds now
 - *Delaying Development Expense.* When a system is retired, all of the system's technical debt is retired with it. Consequently near the end of a system's service life it becomes increasingly difficult to cost-justify investing in anything other than what's most expedient ”



Good and Bad Technical Debt

- “Some debt is taken on in large chunks:
 - “We don't have time to implement this the right way; just hack it in and we'll fix it after we ship.” Conceptually this is like buying a car—it's a large debt that can be tracked and managed”
- Other debt accumulates from taking hundreds or thousands of small shortcuts
 - Generic variable names, sparse comments, creating one class in a case where you should create two, not following coding conventions, and so on
 - This kind of debt is like credit card debt. It's easy to incur unintentionally, it adds up faster than you think, and it's harder to track and manage after it has been incurred
- Both are in response to the directive to “Get it out the door as quickly as possible”
- However, the second kind doesn't pay off even in the short term of an initial development cycle and should be avoided”



Debt Service

- “One of the important implications of technical debt is that it must be *serviced*, i.e., once you incur a debt there will be interest charges
- Business staff generally seems to have a higher tolerance for technical debt than technical staff
 - Business executives tend to want to understand the tradeoffs involved
 - Technical staff seem to believe that the only correct amount of technical debt is *zero*
- The main issue seems to be that technical debt is much less visible than financial debt, and so people have an easier time ignoring it”



Tracking Technical Debt - Options

- “Maintain a debt list within its defect tracking system
 - Each time a debt is incurred, the tasks needed to pay off that debt are entered into the system along with an estimated effort and schedule
 - The debt backlog is then tracked, and any unresolved debt more than 90 days old is treated as critical
- Maintain a debt list as part of its Scrum product backlog, with similar estimates of effort required to pay off each debt
- Either of these approaches can be used to increase visibility into the debt load and into the debt service work that needs to occur within or across release cycles
- Each also provides a useful safeguard against accumulating the "credit card debt" of a mountain of tiny shortcuts mentioned earlier”



Communicating Technical Debt

- *"Use an organization's maintenance budget as a rough proxy for its technical debt service load"*
 - However you will need to differentiate between maintenance that keeps a production system running vs. maintenance that extends the capabilities of a production system. Only the first category counts as technical debt
- *Discuss debt in terms of money rather than in terms of features. For example:*
 - "40% of our current R&D budget is going into supporting previous releases" or "We're currently spending \$2.3 million per year servicing our technical debt"
- *Be sure you're taking on the right kind of debt.*
 - Not all debts are equal. Some debts are the result of good business decisions; others are the result of sloppy technical practices or bad communication about what debt the business intends to take on"

<http://blogs.construx.com/blogs/stevemcc/archive/2007/12/12/technical-debt-decision-making.aspx>



Technical Debt Summary

- “Unintentional Debt. Debt incurred unintentionally due to low quality work
- Intentional Debt. Debt incurred intentionally
 - Short-Term Debt, usually incurred reactively, for tactical reasons
 - Focused Short-Term Debt, individually identifiable shortcuts (like a car loan)
 - Unfocused Short-Term Debt, numerous tiny shortcuts (like credit card debt)
 - Long-Term Debt, usually incurred proactively, for strategic reasons”



Why Technical Debt is Important to You

- The answer to “Is Quality an Accident” has a huge influence on the Unintentional Technical Debt



Summary

- Your decisions and behavior every day define who you are as:
 - An individual on a team
 - A team leader
 - A team
- It is up to you to use quality processes, or not
- It is up to you whether the product you are working on, or the project's infrastructure, incurs too much technical debt